

8. Software Configuration Management

(3 hours, 4 marks)

By Bidur Sapkota

8.1 Software Configuration Management

Define configuration management and explain SCM activities. [4 marks]

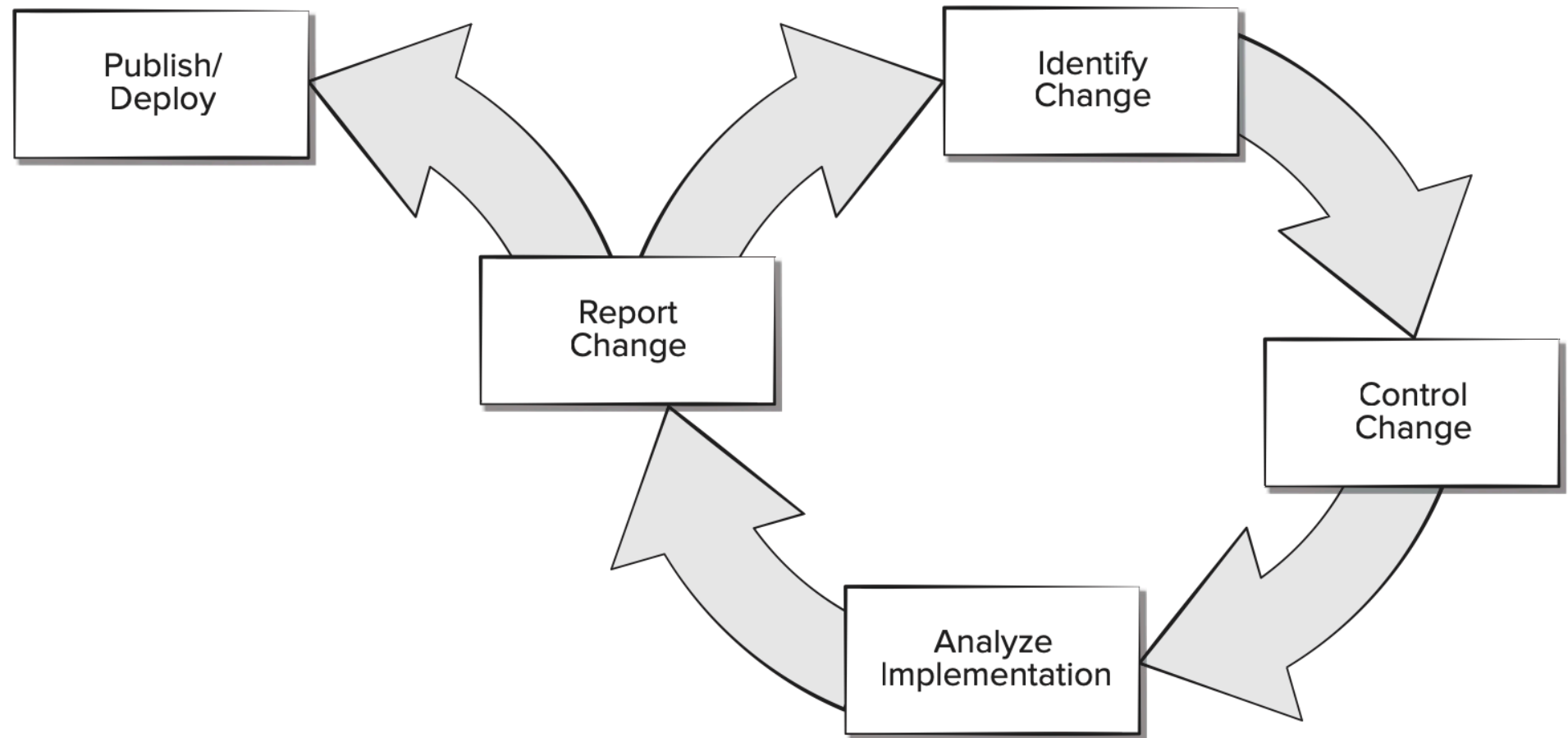
(2080 Bhadra - IOE - Old Syllabus Relevant)

Explain why we need Software Configuration Management (SCM). [4 marks] (2073 Shrawan - IOE - Old Syllabus Relevant)

Software configuration management (SCM) is a set of activities designed to manage change throughout the life cycle of computer software. The output of the software process, which includes programs, documents, and data, collectively forms the software configuration. Because change can occur at any time during development and maintenance, SCM is an umbrella activity applied from the moment a project begins until the software is taken out of operation.

8.1 Software Configuration Management

Software configuration management work flow



8.1 Software Configuration Management

Software configuration management (SCM) is an umbrella activity that is applied throughout the software process. Typical SCM work flow is shown in Figure.

Because change can occur at any time, SCM activities are developed to (1) identify change, (2) control change, (3) ensure that change is being properly implemented, and (4) report changes to others who may have an interest.

8.1 Software Configuration Management

Importance of SCM:

Software projects produce a large number of work products (requirements documents, design models, source code, test plans, user manuals, etc.). These work products are interdependent because a change to one can affect many others.

Without SCM, uncontrolled changes lead to confusion, quality degradation, and delayed delivery. SCM provides the discipline and tools to identify, control, track, and report changes so that the software team can accommodate change without introducing chaos.

8.1 Software Configuration Management

Four fundamental sources of change:

1. New business or market conditions dictate changes in product requirements or business rules.
2. New stakeholder needs demand modification of data, functionality, or services.
3. Reorganization or business growth/downsizing causes changes in project priorities or team structure.
4. Budgetary or scheduling constraints cause a redefinition of the system or product.

8.1 Software Configuration Management

Software Configuration Items (SCIs):

A software configuration item is a named element of information created as part of the software engineering process. An SCI can be as small as a single UML diagram or as large as the complete design document. Examples include specifications, design models, source code files, test plans, test cases, and user documentation. SCIs are organized into configuration objects that are cataloged in the project database with a single name, attributes, and relationships to other objects.

8.1 Software Configuration Management

Baselines:

A baseline is a specification or product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures (IEEE definition).

Before an SCI becomes a baseline, changes can be made quickly and informally.

Once a baseline is established, a specific, formal procedure must be applied to evaluate and verify each change. Common software baselines include:

- 1. System Specification:** This is established after system requirements are reviewed and approved.

8.1 Software Configuration Management

- 2. Software Requirements Specification:** This is established after requirements are reviewed and approved.
- 3. Design Specification:** This is established after the design model is reviewed and approved.
- 4. Source Code:** This is established after code is reviewed, compiled, and tested.
- 5. Test Plans/Procedures/Data:** These are established after test documentation is reviewed.
- 6. Operational System:** This is established after the system is delivered and accepted.

8.1 Software Configuration Management

When a member of the team wants to modify a baselined SCI, it is copied from the project database into the engineer's private workspace. The modification can proceed only if SCM controls are followed. After modification, the SCI is checked back in through the controlled process.

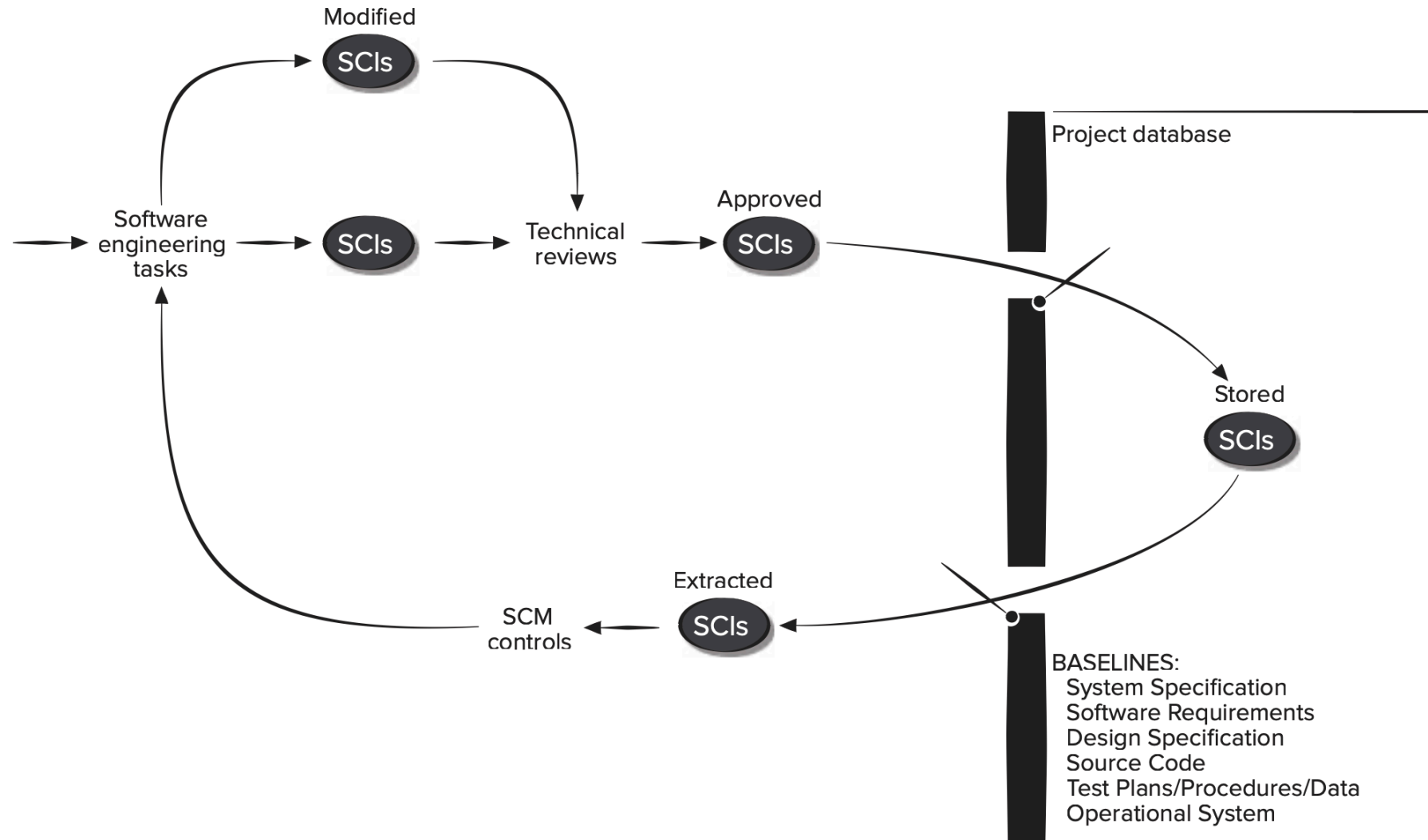
8.1 Software Configuration Management

Elements of a Configuration Management System:

- 1. Component elements:** A set of tools coupled within a file management system (e.g., a database/repository) that enables access to and management of each SCI.
- 2. Process elements:** A collection of procedures and tasks that define an effective approach to change management for all constituencies.
- 3. Construction elements:** A set of tools that automate the construction of software by ensuring that the proper set of validated components (correct version) have been assembled.
- 4. Human elements:** A set of tools and process features used by the software team to implement effective SCM.

8.1 Software Configuration Management

FIGURE 22.2 Baselined SCIs and the project database



8.1 Software Configuration Management

SCM Activities (Core Tasks):

SCM tasks can be viewed as concentric layers through which SCIs flow throughout their useful life:

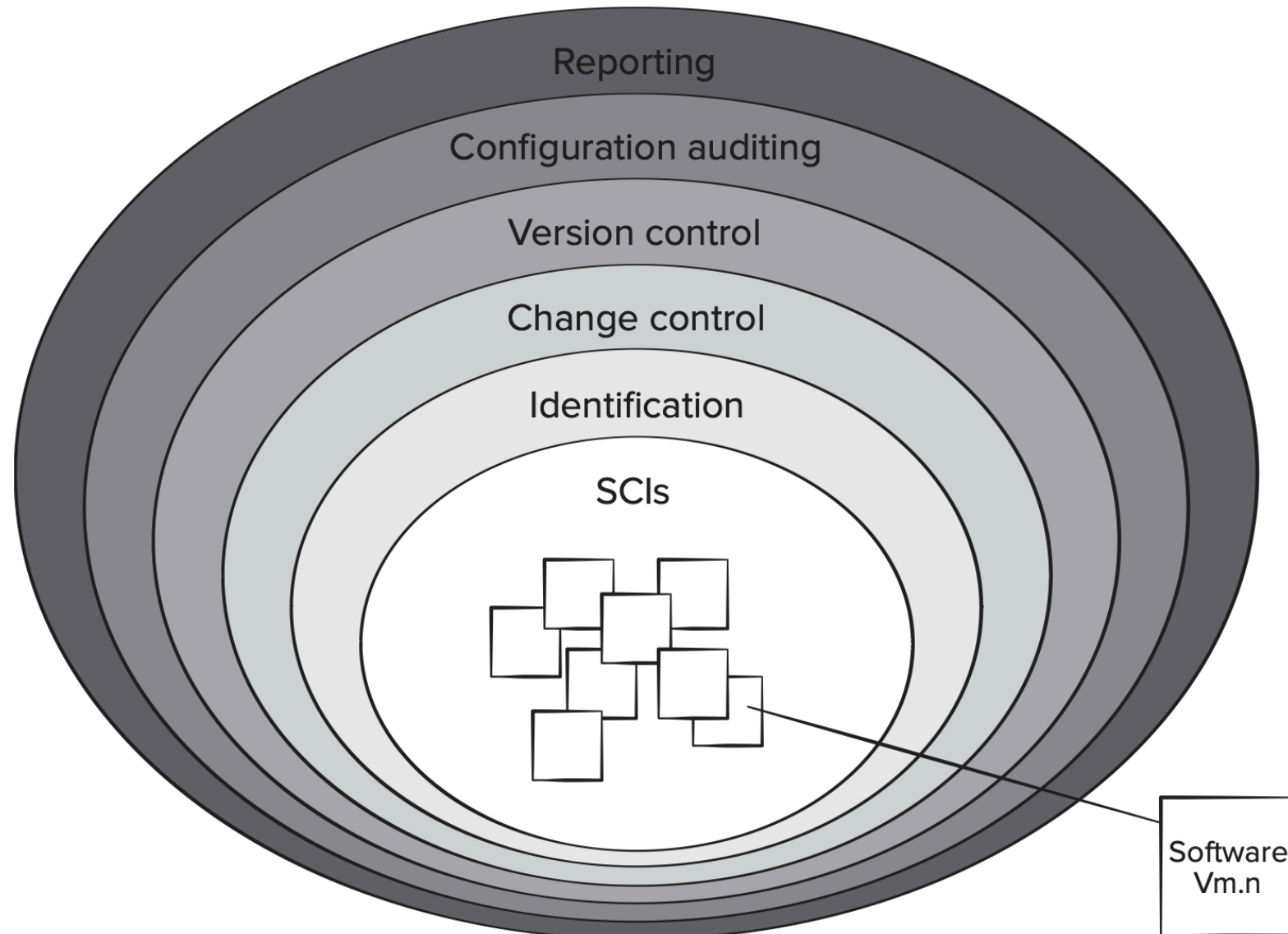
- 1. Identification:** Uniquely naming and describing every SCI so it can be tracked and managed.
- 2. Change control:** Combining human procedures and automated tools to provide a mechanism for controlling change requests, evaluating them, and implementing approved changes.
- 3. Version control:** Managing different versions and variants of configuration objects created during the software process.

8.1 Software Configuration Management

- 4. Configuration auditing:** Ensuring that changes have been properly implemented and that quality is maintained.
- 5. Status reporting (status accounting):** Recording and reporting information about each change, including what happened, who did it, when it happened, and what else will be affected.

8.1 Software Configuration Management

Layers of the SCM process



8.2 Version Control and Continuous Integration

Define version, variants, and release with respect to configuration management. [4 marks] (2079 Bhadra - IOE - Old Syllabus Relevant)

Differentiate version and variant in the context of configuration management. [2 marks] (2082 Bhadra - IOE - Old Syllabus Relevant)

Version Control

Version control combines procedures and tools to manage different versions of configuration objects created during the software process. A version control system implements four major capabilities:

1. Project database (repository): Stores all relevant configuration objects.

8.2 Version Control and Continuous Integration

- 2. Version management capability:** Stores all versions of a configuration object (or enables any version to be constructed using differences from past versions).
- 3. Make/build facility:** Collects all relevant configuration objects and constructs a specific version of the software.
- 4. Issue tracking (bug tracking):** Records and tracks the status of all outstanding issues associated with each configuration object.

8.2 Version Control and Continuous Integration

Version, Variant, and Release:

Version: A version is a specific, identifiable state of a software configuration item at a particular point in time. Versions are sequential and represent the evolution of a product over time (e.g., v1.0, v1.1, v2.0). Each new version typically incorporates bug fixes, enhancements, or new features relative to the previous version. The evolution of a program can be tracked by examining the revision history of all configuration objects.

8.2 Version Control and Continuous Integration

Variant: A variant is a parallel version of a software product designed to serve different needs, markets, or hardware/software configurations. Unlike versions, which are sequential, variants exist in parallel. For example, a software product might have a "Windows" variant and a "Linux" variant, or a "Standard" edition and a "Enterprise" edition, maintained simultaneously.

Release: A release is a specific version (or set of versions of components) that has been officially approved, packaged, and distributed to customers or end users. While there are many internal versions during development, only selected, stable versions are formally released. A release is typically tagged with a release number and accompanied by release notes.

8.2 Version Control and Continuous Integration

Difference between version and variant: A version represents sequential evolution of the same product over time, such as progressing from v1.0 to v1.1 and then to v2.0. A variant represents parallel, simultaneous forms of the same product tailored for different environments, platforms, or customer segments. All variants may share the same version number but differ in configuration.

Change Set: A change set is a collection of all changes (to some baseline configuration) required to create a specific version of the software. It captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when. Named change sets can be identified and applied to a baseline configuration to construct any specific version.

8.2 Version Control and Continuous Integration

Access control and synchronization control:

- **Access control** governs which software engineers have the authority to access and modify a particular configuration object.
- **Synchronization control** ensures that parallel changes performed by two different people don't overwrite one another. Modern version control systems handle this through locking, merging, or branching mechanisms.

8.2 Version Control and Continuous Integration

Continuous Integration

Continuous Integration (CI) is the practice of merging all developers' working copies to a shared mainline frequently, which is often done multiple times per day. Each integration triggers an automated build and test process to detect errors as quickly as possible.

How CI works:

1. A developer commits code changes to the shared repository.
2. The CI server (e.g., Jenkins, GitHub Actions, GitLab CI, Travis CI) automatically detects the change.
3. The server builds the project and runs the automated test suite.

8.2 Version Control and Continuous Integration

4. If any test fails, the pipeline halts and the developer is notified immediately.
5. Only code that passes all tests is promoted to the next stage.

Advantages of CI:

- **Accelerated feedback:** Notifying developers immediately when integration fails allows fixes to be made while the number of performed changes is small.
- **Increased quality:** Building and integrating software whenever necessary provides confidence in the quality of the developed product.
- **Reduced risk:** Integrating components early avoids risking a long, painful integration phase because design failures are discovered and fixed early.

8.2 Version Control and Continuous Integration

- **Improved reporting:** Providing additional information (e.g., code analysis metrics) allows for more accurate configuration status accounting.
- **Early defect capture:** Always reduces development costs by allowing cheaper fixes earlier in the project timeline.

Best practices for SCM with CI:

1. Keep the number of code variants small.
2. Test early and often.
3. Integrate early and often.
4. Use tools to automate testing, building, and code integration.

8.3 Change Management Process

Describe the change management process in software engineering. [4 marks] (2078 Bhadra - IOE - Old Syllabus Relevant)

The change management process defines a series of tasks with four primary objectives: (1) identify all items that collectively define the software configuration, (2) manage changes to one or more of these items, (3) facilitate the construction of different versions of an application, and (4) ensure that software quality is maintained as the configuration evolves over time.

8.3 Change Management Process

The Change Control Process:

- 1. Change request submission:** A user, developer, or stakeholder submits a formal change request describing the desired modification.
- 2. Evaluation:** The developer or a designated evaluator assesses the change request for technical merit, potential side effects, overall impact on other configuration objects and system functions, and the projected cost of the change.
- 3. Change report:** The results of the evaluation are presented as a change report.

8.3 Change Management Process

- 4. Change Control Authority (CCA) decision:** The change report is reviewed by a Change Control Authority, which is a person or group that makes a final decision on the status and priority of the change. The CCA may be the project manager alone (on small projects) or a committee with representatives from software, hardware, database engineering, support, and marketing. The CCA takes a global view, assessing the impact of the change beyond the immediate SCI.
- 5. Engineering Change Order (ECO):** If the change is approved, an ECO is generated describing the change to be made, the constraints that must be respected, and the criteria for review and audit. If the change is rejected, the user is informed with reasons.

8.3 Change Management Process

- 6. Check out:** The affected configuration objects are checked out from the repository into the developer's workspace.
- 7. Implementation:** The developer makes the approved change.
- 8. Review and audit:** The change is reviewed (technical review) and audited (configuration audit) to verify correctness, completeness, and compliance with standards.
- 9. Check in:** The modified objects are checked back into the repository using version control mechanisms, creating a new version.

8.3 Change Management Process

10. Baseline and testing: A new baseline is established for testing. QA and testing activities are performed.

11. Release: The changes are promoted and included in the next release. The new software version is rebuilt and distributed.

8.3 Change Management Process

Levels of change control:

- **Before baseline:** Only informal change control is needed. The developer may make changes freely as justified by project and technical requirements.
- **After baseline (project level):** Project-level change control is implemented. The developer must gain approval from the project manager (for local changes) or from the CCA (for changes affecting other SCIs).
- **After release to customers:** Formal change control is instituted with the full process described above.

8.3 Change Management Process

Configuration Audit:

A software configuration audit complements technical reviews by assessing a configuration object for characteristics not typically considered during review. The audit verifies:

1. Has the change specified in the ECO been made? Have any additional unauthorized modifications been incorporated?
2. Has a technical review been conducted to assess technical correctness?
3. Has the software process been followed, and have standards been properly applied?

8.3 Change Management Process

4. Has the change been highlighted in the SCI? Have the change date and author been specified?
5. Have SCM procedures for noting, recording, and reporting the change been followed?
6. Have all related SCIs been properly updated?

8.3 Change Management Process

Configuration Status Reporting (CSR):

Status reporting answers: (1) What happened? (2) Who did it? (3) When did it happen? (4) What else will be affected? A CSR entry is made each time an SCI is assigned new or updated identification, each time a change is approved by the CCA, and each time a configuration audit is conducted. CSR output may be placed in an online database or website so that developers and support staff can access change information.

8.3 Change Management Process

Impact Management:

Before implementing a change, impact analysis must be performed to understand the web of software work product interdependencies. Impact management involves:

- **Impact network:** Identifies team members and stakeholders who might affect or be affected by changes.
- **Forward impact management:** Assesses the impact of your changes on others and informs affected members.
- **Backward impact management:** Examines changes made by others and their impact on your work, incorporating mechanisms to mitigate the impact.

8.4 Branching Strategies (Git-flow Basics)

Git-flow is a structured branching model introduced by Vincent Driessen in 2010. It assigns specific roles to different branches and defines how and when they should interact. Git-flow is well suited for projects with scheduled release cycles and teams that need clear separation between development, release preparation, and production hotfixes.

Core Branches (permanent):

1. `main` (or `master`):

The main branch stores the official production-ready release history. Every commit on main represents a stable, deployable version of the software. Commits on main are typically tagged with a version number (e.g., v1.0.0, v2.1.0). No one develops directly on main.

8.4 Branching Strategies (Git-flow Basics)

2. **develop** :

The develop branch serves as the integration branch for features. It contains the latest delivered development changes for the next release. All new features are merged into develop. When develop reaches a stable point and is ready for release, it is merged into main through a release branch.

8.4 Branching Strategies (Git-flow Basics)

Supporting Branches (temporary):

3. **feature/*** branches:

- **Purpose:** Used for developing new features or functionality.
- **Branched from:** `develop`.
- **Merged back into:** `develop`.
- **Naming convention:** `feature/login-page`, `feature/payment-gateway`.
- **Workflow:** A developer creates a feature branch from `develop`, works on the feature in isolation, and once the feature is complete and tested, merges it back into `develop` via a pull request. The feature branch is then deleted.

8.4 Branching Strategies (Git-flow Basics)

4. `release/*` branches:

- **Purpose:** Used to prepare for a new production release (final bug fixes, documentation updates, version number bumps).
- **Branched from:** `develop`.
- **Merged back into:** Both `main` and `develop`.
- **Naming convention:** `release/1.2.0`.
- **Workflow:** When `develop` has acquired enough features for a release, a release branch is created. No new features are added on this branch. It only contains bug fixes, documentation updates, and release-oriented tasks. Once ready, it is merged into `main` (and tagged with a version number) and also merged back into `develop` so that any fixes made during release preparation are not lost.

8.4 Branching Strategies (Git-flow Basics)

5. **hotfix/*** branches:

- **Purpose:** Used for urgent, unplanned fixes that must go directly into production.
- **Branched from:** `main`.
- **Merged back into:** Both `main` and `develop`.
- **Naming convention:** `hotfix/critical-bug-fix`.
- **Workflow:** When a critical bug is discovered in production, a hotfix branch is created directly from `main`. The fix is implemented and tested. Once complete, the hotfix branch is merged into `main` (and tagged with an updated version number) and also merged into `develop` to ensure the fix is included in ongoing development.

8.4 Branching Strategies (Git-flow Basics)

Basic Git Operations in Git-flow:

- **git commit** : Records changes to the local repository with a descriptive message.
- **git branch <name>** : Creates a new branch from the current branch.
- **git checkout <branch>** (or **git switch <branch>**): Switches to the specified branch.
- **git merge <branch>** : Merges the specified branch into the current branch.
- **git pull** : Fetches and integrates changes from the remote repository.
- **git push** : Uploads local commits to the remote repository.

8.4 Branching Strategies (Git-flow Basics)

Conflict Resolution:

When two developers modify the same lines in the same file on different branches, a merge conflict occurs during merge. Git marks the conflicting sections in the file. The developer must manually review the conflicting changes, decide which changes to keep (or combine both), edit the file to resolve the conflict, and then commit the resolution.